SymPy Bug Report

1 Bug 54: singularities misses the pole of gamma(x) at zero

1.1 Minimal Reproducer

```
from sympy import S, gamma, residue, series, symbols
from sympy.calculus.singularities import singularities

x = symbols("x")
s = singularities(gamma(x), x, S.Complexes)

print("singularities =", s)
print("contains 0 =", s.contains(0))
print("gamma(0) =", gamma(0))
print("residue at 0 =", residue(gamma(x), x, 0))
print("series at 0 =", series(gamma(x), x, 0, 1))
```

1.2 Observed Behavior

```
singularities = EmptySet
contains 0 = False
gamma(0) = zoo
residue at 0 = 1
series at 0 = 1/x - EulerGamma + O(x)
```

The singularity query reports no singularities and specifically reports that the returned set does not contain 0. This contradicts the same symbolic system's direct evidence that $\Gamma(0)$ is infinite, that the residue at 0 is 1, and that the Laurent expansion has a $1/x$ term.

1.3 Expected Behavior

The result of singularities$(\Gamma(x), x, \mathbb{C})$ should contain 0. More completely, the finite poles of $\Gamma(x)$ occur at the nonpositive integers, so a correct result should include

$$\{0, -1, -2, -3, \ldots\}.$$

1.4 Mathematical Explanation

The gamma function $\Gamma(x)$ is meromorphic with simple poles at every nonpositive integer. At the origin its Laurent expansion is

$$\Gamma(x) = \frac{1}{x} - \gamma_{\mathrm{E}} + O(x),$$

where γ_{E} is Euler's constant. The coefficient of $1/x$ is nonzero, so $x = 0$ is a genuine pole, not a removable singularity or a branch-convention artifact. Therefore $\emptyset$ is not an equivalent representation of the singularity set: it asserts that there are no singularities in the requested complex domain, while 0 is necessarily in that set.

1.5 Source-Level Diagnosis

The diagnosis localizes the miss to `sympy/calculus/singularities.py`, specifically the scanner in `singularities` around lines 94–108. The traced public call path enters the `singularities` function in the `sympy.calculus.singularities` module, sympifies $\Gamma(x)$, rewrites only selected trigonometric and hyperbolic reciprocal forms, scans `Pow` atoms, and then scans only the atoms `log`, `asech`, `acsch`, `atanh`, and `acoth`. For $\Gamma(x)$, those scans do not generate any singularity conditions, so the initially empty set is returned unchanged.

The diagnosis also notes that `sympy/functions/special/gamma_functions.py` contains related gamma-function knowledge: `gamma.eval` evaluates nonpositive integer numeric arguments to `ComplexInfinity`, and the class metadata lists `ComplexInfinity` in `_singularities`. However, `singularities()` does not use that evaluation behavior to derive finite symbolic poles. As a result, the symbolic scanner treats $\Gamma(x)$ as opaque and never constructs the condition that the argument of Γ lies in the nonpositive integers. A plausible fix direction identified by the diagnosis is to add a special-function path for $\Gamma(g(x))$, solving $g(x) \in \{0, -1, -2, \ldots\}$, or to use richer function-level meromorphic singularity metadata.

1.6 Additional Failing Instances

The parametrized evidence checks shifted gamma functions of the form $\Gamma(x + s)$. In each case, Γ has a pole whenever $x + s$ is a nonpositive integer; in particular, $x = -s$ makes the argument equal to 0. The current behavior omits that representative pole for every tested shift.

Input / Expression	**SymPy behavior**	**Expected behavior**
$\mathrm{singularities}(\Gamma(x), x, \mathbb{C})$	Returned set does not contain 0	Returned set contains 0
$\mathrm{singularities}(\Gamma(x+1), x, \mathbb{C})$	Returned set does not contain -1	Returned set contains -1
$\mathrm{singularities}(\Gamma(x+2), x, \mathbb{C})$	Returned set does not contain -2	Returned set contains -2
$\mathrm{singularities}(\Gamma(x-1), x, \mathbb{C})$	Returned set does not contain 1	Returned set contains 1
$\mathrm{singularities}(\Gamma(x+3), x, \mathbb{C})$	Returned set does not contain -3	Returned set contains -3
$\mathrm{singularities}(\Gamma(x-2), x, \mathbb{C})$	Returned set does not contain 2	Returned set contains 2

These are the same mathematical failure rather than unrelated edge cases: the scanner fails to propagate the finite pole set of Γ through a simple affine argument.

1.7 Related Issues Assessment

The best-effort deduplication check found public material related to gamma poles and singularities, including SymPy issue 14567 about a `ValueError` involving gamma-function poles while printing an

expression with `factorial`, and issue 18455 about better singularity handling in plots. The available evidence does not identify a public report that `singularities(gamma(x), x, S.Complexes)` silently returns `EmptySet` or omits the finite pole at 0. The deduplication verdict is therefore a new failure mode with no public precedent, and the case remains individually actionable as a precise calculus-special-functions regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import S, gamma, symbols
from sympy.calculus.singularities import singularities


@pytest.mark.parametrize("shift", [0, 1, 2, -1, 3, -2])
def test_singularities_gamma_shifted_poles(shift):
    x = symbols("x")
    assert singularities(gamma(x + shift), x, S.Complexes).contains(-shift) == S.true
```